```
import asyncio
import json
import logging
import os
import threading
from datetime import datetime
from flask import Flask

from aiogram import Bot, Dispatcher, F, Router
from aiogram.client.default import DefaultBotProperties
from aiogram.enums import ParseMode
from aiogram.filters import CommandStart, Command
from aiogram.types import (
    Message, CallbackQuery,
    InlineKeyboardMarkup, InlineKeyboardButton
)

# --- НАСТРОЙКИ (ТОКЕН БЕРЁТСЯ ИЗ ПЕРЕМЕННОЙ ОКРУЖЕНИЯ RENDER) ---
BOT_TOKEN =
os.environ.get("8625485347:AAFVktB6PVRxDrFIMnNC_whjnoouVKA71mc",
"СЮДА_ВСТАВЬТЕ_ТОКЕН_ЕСЛИ_ТЕСТИТЕ_ЛОКАЛЬНО")
ADMIN_IDS = [1814124100] # ВАШ ID
REVEAL_AFTER = 5
DATA_FILE = "data.json"
# ------------------------------------------------------------

# --- ВЕБ-СЕРВЕР ДЛЯ RENDER (ЧТОБЫ ОН НЕ РУГАЛСЯ) ---
app = Flask(__name__)

@app.route('/')
@app.route('/health')
def health():
    return "Bot is running", 200

def run_flask():
    # Render сам назначает порт через переменную PORT
    port = int(os.environ.get("PORT", 10000))
    app.run(host="0.0.0.0", port=port)

# --- ВАШ КОД БОТА (БЕЗ ИЗМЕНЕНИЙ) ---
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
%(message)s')
router = Router()

def load_data():
    default_data = {"users": {}, "msg_counter": {}, "banned": [], "reveal_all": False,
"messages": {}}
    if os.path.exists(DATA_FILE):
```

```
        try:
            with open(DATA_FILE, "r", encoding="utf-8") as f:
                loaded = json.load(f)
                for k, v in default_data.items():
                    if k not in loaded: loaded[k] = v
                return loaded
        except Exception: pass
    return default_data

def save_data():
    try:
        with open(DATA_FILE, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=2)
    except Exception: pass

data = load_data()
last_incoming = {}
reply_map = {}

def is_admin(user_id): return user_id in ADMIN_IDS
def is_banned(user_id): return user_id in data.get("banned", [])

async def register_user(user, bot):
    uid = str(user.id)
    now = datetime.now().strftime("%Y-%m-%d %H:%M")
    if uid not in data["users"]:
        data["users"][uid] = {"name": user.full_name, "username": user.username or "",
"first_seen": now, "last_seen": now}
        save_data()
        for admin_id in ADMIN_IDS:
            try: await bot.send_message(admin_id, f"🆕 Новый: {user.full_name}
(<code>{uid}</code>)")
            except: pass
    else:
        data["users"][uid]["last_seen"] = now
        save_data()

@router.message(CommandStart(deep_link=True))
async def start_with_link(message: Message, bot: Bot):
    if is_banned(message.from_user.id): return await message.answer("⛔ Бан.")
    await register_user(message.from_user, bot)
    try:
        payload = message.text.split(maxsplit=1)[1]
        recipient_id = int(payload)
        if recipient_id == message.from_user.id: return await message.answer("Себе нельзя
🙂")
        last_incoming[message.from_user.id] = recipient_id
```

```
        await message.answer(f"✉️ Пиши анонимно. После {REVEAL_AFTER} сообщений тебя раскроют.")
    except: await message.answer("Некорректная ссылка.")

@router.message(CommandStart())
async def start(message: Message, bot: Bot):
    if is_banned(message.from_user.id): return await message.answer("⛔ Бан.")
    await register_user(message.from_user, bot)
    me = await bot.get_me()
    await message.answer(f"👋 Твоя ссылка:\nhttps://t.me/{me.username}?start={message.from_user.id}")

def admin_main_kb():
    status = "🔓 ВКЛ" if data.get("reveal_all") else "🔒 ВЫКЛ"
    return InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text=f"👁️ Раскрывать всех: {status}", callback_data="adm:toggle")],
        [InlineKeyboardButton(text="👥 Юзеры", callback_data="adm:users")],
        [InlineKeyboardButton(text="📊 Статы", callback_data="adm:stats")],
    ])

@router.message(Command("admin"))
async def admin_cmd(message: Message):
    if is_admin(message.from_user.id): await message.answer("⚙️ Админка", reply_markup=admin_main_kb())

@router.callback_query(F.data.startswith("adm:"))
async def admin_callback(callback: CallbackQuery):
    if not is_admin(callback.from_user.id): return await callback.answer("Нет доступа", show_alert=True)
    action = callback.data.split(":")[1]
    if action == "toggle":
        data["reveal_all"] = not data.get("reveal_all", False); save_data()
        await callback.message.edit_reply_markup(reply_markup=admin_main_kb()); await callback.answer("Ок")
    elif action == "users": await callback.message.answer(f"👥 Всего: {len(data['users'])}"); await callback.answer()
    elif action == "stats": await callback.message.answer(f"📊 Сообщений: {sum(data['msg_counter'].values())}"); await callback.answer()

@router.message()
async def forward_message(message: Message, bot: Bot):
    if is_banned(message.from_user.id): return await message.answer("⛔ Бан.")
    await register_user(message.from_user, bot)
    sender_id = message.from_user.id
    if message.reply_to_message and message.reply_to_message.from_user.id == (await bot.get_me()).id:
        target = reply_map.get(message.reply_to_message.message_id)
```

```
        if target: return await send_content(bot, target, message, reply_to=sender_id)
    recipient_id = last_incoming.pop(sender_id, None)
    if not recipient_id: return await message.answer("Сначала перейди по чьей-то ссылке или получи свою: /link")
    sent = await send_content(bot, recipient_id, message, reply_to=sender_id)
    if sent: await message.answer("✅ Отправлено!")

async def send_content(bot: Bot, recipient_id: int, message: Message, reply_to: int):
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="↩️ Ответ", callback_data=f"reply:{reply_to}")],
        [InlineKeyboardButton(text="🔓 Раскрыть", callback_data=f"reveal:{reply_to}")]
    ])
    user = message.from_user
    header = f"<b>👤 От:</b> {user.full_name} (<code>{user.id}</code>)\n\n" if data.get("reveal_all") else "📩 <b>Анонимно:</b>\n\n"
    try:
        if message.text: sent = await bot.send_message(recipient_id, f"{header}{message.text}", reply_markup=kb)
        elif message.photo: sent = await bot.send_photo(recipient_id, message.photo[-1].file_id, caption=f"{header}{message.caption or ''}", reply_markup=kb)
        elif message.video: sent = await bot.send_video(recipient_id, message.video.file_id, caption=f"{header}{message.caption or ''}", reply_markup=kb)
        else: return await message.answer("Только текст, фото, видео.")
        reply_map[sent.message_id] = reply_to
        key = f"{reply_to}:{recipient_id}"; data["msg_counter"][key] = data["msg_counter"].get(key, 0) + 1
        data["messages"][str(sent.message_id)] = {"sender_id": user.id, "name": user.full_name, "username": user.username or "", "count": data["msg_counter"][key]}
        save_data(); return sent
    except: return None

@router.callback_query(F.data.startswith("reply:"))
async def reply_button(callback: CallbackQuery):
    last_incoming[callback.from_user.id] = int(callback.data.split(":")[1])
    await callback.message.answer("✍️ Пиши ответ:"); await callback.answer()

@router.callback_query(F.data.startswith("reveal:"))
async def reveal_button(callback: CallbackQuery):
    info = data["messages"].get(str(callback.message.message_id))
    if not info: return await callback.answer("Утеряно.", show_alert=True)
    if not is_admin(callback.from_user.id) and info["count"] < REVEAL_AFTER:
        return await callback.answer(f"Раскрытие после {REVEAL_AFTER}. Написано: {info['count']}", show_alert=True)
    await callback.message.answer(f"👤 <b>Отправитель:</b>\n{info['name']}\nID: <code>{info['sender_id']}</code>\nСообщений: {info['count']}")
    await callback.answer("Готово")

# --- ЗАПУСК БОТА В ФОНЕ ---
```

```
async def main_bot():
    bot = Bot(token=BOT_TOKEN,
default=DefaultBotProperties(parse_mode=ParseMode.HTML))
    dp = Dispatcher(); dp.include_router(router)
    await bot.delete_webhook(drop_pending_updates=True)
    await dp.start_polling(bot)

def run_bot():
    asyncio.run(main_bot())

if __name__ == "__main__":
    # Запускаем Flask (для Render) в отдельном потоке
    threading.Thread(target=run_flask, daemon=True).start()
    # Запускаем бота в основном потоке
    print("Бот запускается...")
    run_bot()
```